appendix A. Microservices architecture best practices

In chapter 2, we explained some of the best practices that we should consider when creating a microservice. With this appendix we want to go deeper into these best practices. In the next sections, we will give you some guidelines (or best practices) to create a successful microservices architecture.

It is important to note that there isn't a well-defined set of rules to achieve this; however, Hüseyin Babal in his talk "Ultimate Guide to Microservice Architecture" presented a set of best practices to achieve a flexible, efficient, and scalable design. We consider this talk essential among the developer's community because it not only gives guidelines on how to develop a microservices architecture, but it also highlights which are the fundamental components of a microservice. In this appendix, we have selected some of the practices that we consider most important (those that we used throughout the book to create a flexible and successful architecture).

NOTE If you're interested in knowing all of the best practices mentioned in the "Ultimate Guide to Microservice Architecture" talk, visit the following link that contains a video describing them: **https://www.youtube.com/watch?v=CTnMUE06oZI**.

The following list gives some of the best practices mentioned in his talk. We'll cover these in the following sections.

- Richardson Maturity Model
- Spring HATEOAS
- Distributed configuration
- Continuous delivery

- Monitoring
- Logging

- Application performance management
- API gateways

A.1 Richardson Maturity Model

This best practice (**http://martinfowler.com/articles/richardsonMaturityModel.html**), described by Martin Fowler, is a guide to understanding the main principles of REST architectures and to evaluate our REST architecture. It's important to note that we can see these levels more as a tool to help us understand the components of REST and the ideas behind the RESTful thinking rather than a kind of assessment mechanism for your company. Figure A.1 shows the different levels of maturity used to evaluate a service.

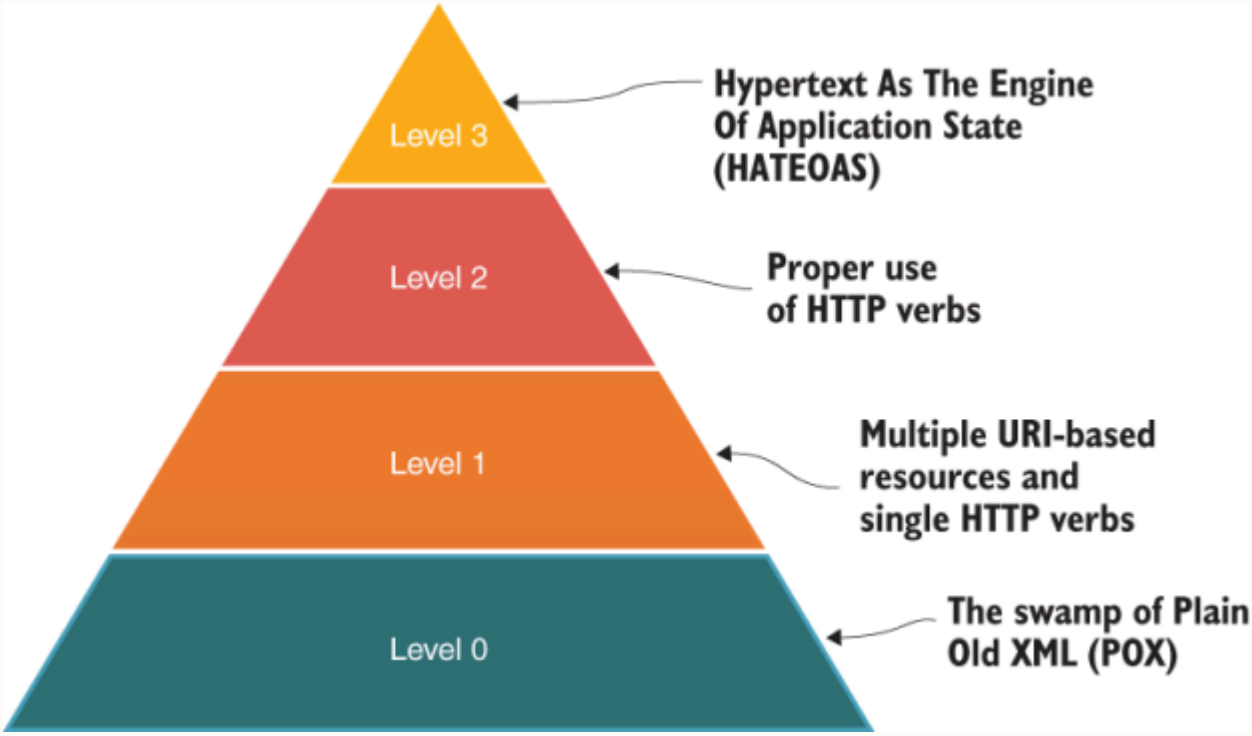


Figure A.1 Richardson's Maturity Model showing the levels of maturity

In the Richardson Maturity Model, level 0 represents the basic functionalities expected of an API. Basically, the API interaction in this level is a simple remote procedure call (RPC) to a single URI. It's mostly a back-and-forth XML sent in every request and response that specifies the action, the target, and the parameters to run the service.

For example, let's assume that we want to buy a specific product from an online store. First, the store's software needs to verify that is has the product in stock. At this level, the online store will expose a service endpoint with a single URI. Figure A.2 shows the level 0 service requests and responses.

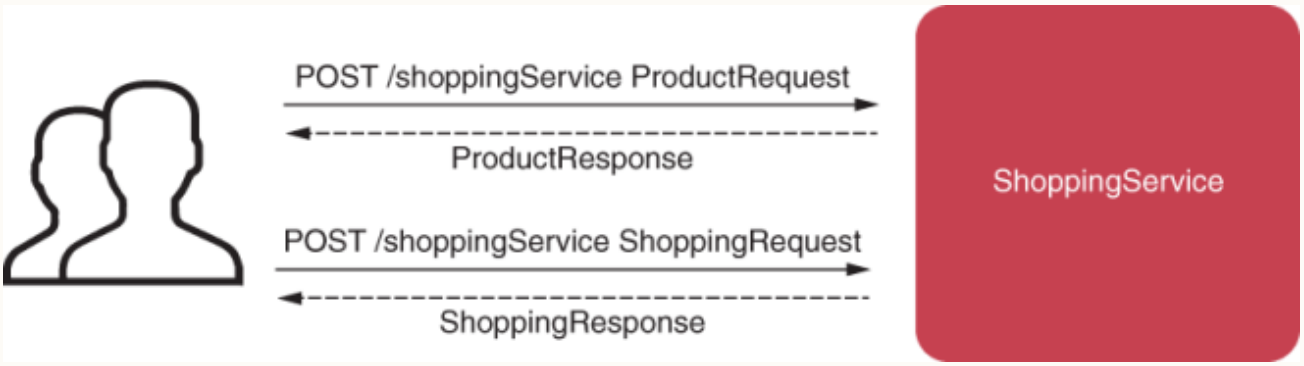


Figure A.2 Level 0: a simple RPC with a plain old XML (POX) sent back and forth between consumer and service

With level 1, the main idea is to perform actions on individual resources rather than making requests to a singular service endpoint. Figure A.3 shows the individual resources with their requests and responses.



Figure A.3 Level 1: individual resource calls to perform some action

At level 2, the services use HTTP verbs in order to perform specific actions. The GET HTTP verb is used to retrieve, POST to create, PUT to update, and DELETE to remove. As previously mentioned, these levels function more as a tool to help us understand the components of REST, and REST advocates mention using all the HTTP verbs. Figure A.4 shows how to implement the use of HTTP verbs in our store example.

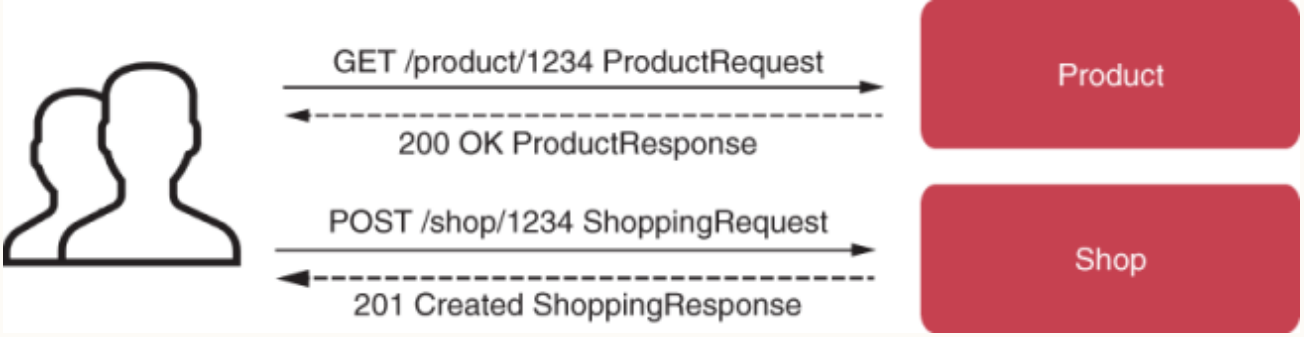


Figure A.4 HTTP verbs. In our scenario, GET retrieves product information and POST creates the shopping order.

The final level, level 3, introduces Hypertext as the Engine of Application State (HATEOAS). By implementing HATEOAS, we can make our API respond with additional information, such as link resources, to create richer interactions (figure A.5).

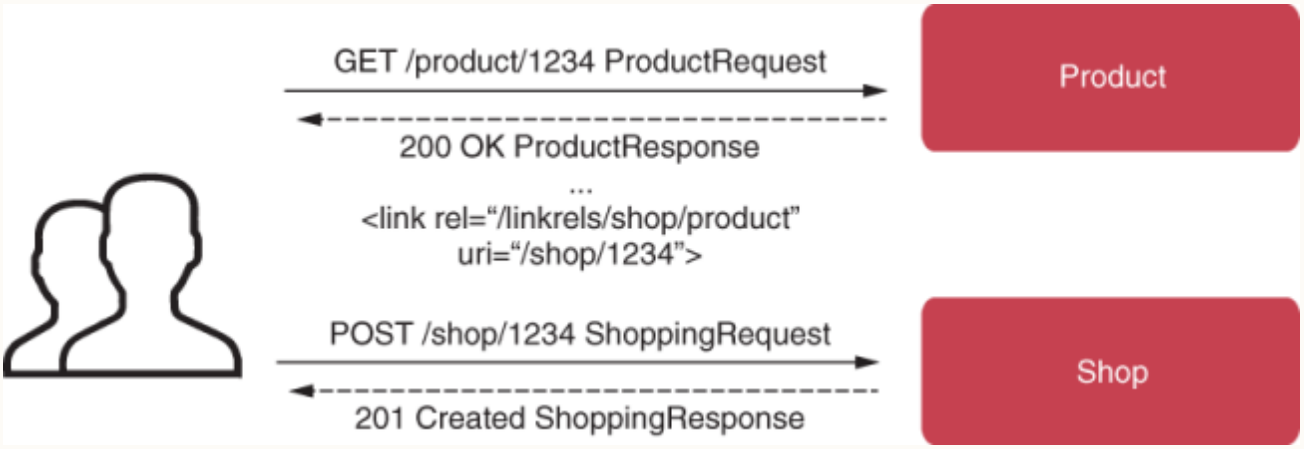


Figure A.5 HATEOAS in the retrieve product response. In our scenario, the link shows additional information on how to buy a specific product.

Using the Richardson Maturity Model, it is easy to explain that there is no definitive way to implement REST-based applications. We can, however, choose the degree of adherence that best suits the needs of our projects.

A.2 Spring HATEOAS

Spring HATEOAS (**https://spring.io/projects/spring-hateoas**) is a small Spring project that allow us to create APIs that follow the HATEOAS principle explained in level 3 of the Richardson Maturity Model. With Spring HATEOAS, you can quickly create model classes for links and resource representations. It also provides a link builder API to create specific links that point to Spring MVC controller methods, and more.

A.3 Externalized configuration

This best practice goes hand in hand with the configuration best practice listed in the twelve-factor app mentioned in chapter two (section 2.3). The configuration of our microservices should never be in the same repository as our source code. Why is it that important?

Microservices architectures are composed of a collection of services (microservices) that run in separate processes. Each service can be deployed and scaled independently, creating more instances of the same microservice. So, let's say that you, as a developer, want to change a configuration file of a specific microservice that has been scaled several times. If you don't follow this best practice and you have the configuration packaged with the deployed microservice, you'll be forced to redeploy each instance. Not only is this time consuming, but it also can lead to configuration issues between the microservices.

There are many ways to implement externalized configurations in a microservice architecture. But in this book, we used Spring Cloud Config as described in chapter 2.

A.4 Continuous integration (CI) and continuous delivery (CD)

Continuous integration (CI) is a series of software development practices where the team members integrate their changes to a repository within a short period of time to detect possible errors and to analyze the software's quality that they created. This is accomplished by using an automatic and continuous code check (build) that includes the execution of tests. On the other hand, continuous delivery (CD) is a software development practice in which the process of delivering software is automated to allow short-term deliveries into a production environment.

When we apply these processes to our microservices architecture, it's essential to keep in mind that there should never be a "waiting list" for integration and release to production. For example, a team in charge of service X must be able to publish changes to production at any time, without having to wait for the release of other services. Figure A.6 shows a high-level diagram of how our CI/CD process should look when working with several teams.

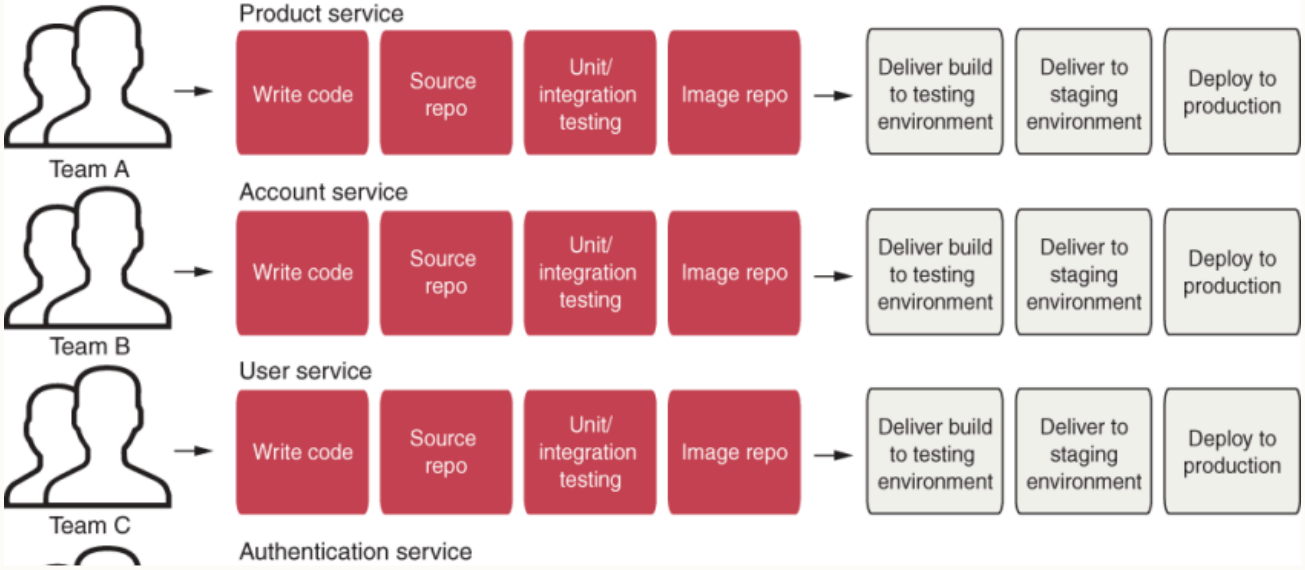


Figure A.6 Microservices CI/CD processes with a test and staging environment. In this figure, each team has its own source repository, unit and integration testing, image repository, and process of deployment. This allows each team to deploy and release newer versions without having to wait for other releases.

For the provisioning implementations, we need to make a technology shift. Spring framework(s) are geared toward application development and don't have tools for creating a build/deployment pipeline.

A.5 Monitoring

The monitoring process is critical when we talk about microservices. Microservices are part of large and complex distributed systems, and remember, the more distributed a system is, the more complicated it is to find and solve its problems. Imagine that we have an architecture with hundreds of microservices, and one of the microservices is affecting the performance of other services. How do we know which is the unstable service if we don't implement a suitable monitoring process? In order to tackle this, in appendix C, we explain how to achieve a good monitoring system in our architecture using the following tools. Figure A.7 shows the interaction between Micrometer, Prometheus, and Grafana.

- Micrometer (**https://micrometer.io/**) is a default metrics library in Spring Boot 2 that allows us to obtain application metrics as well as JVM metrics like garbage collection, memory pools, and so forth.
- Prometheus (**https://prometheus.io/**) is an open source monitoring and alert system that stores all data as a time series.

- Grafana (https://grafana.com/)is an analysis platform for metrics that lets you query, visualize, and understand data no matter where it's stored.

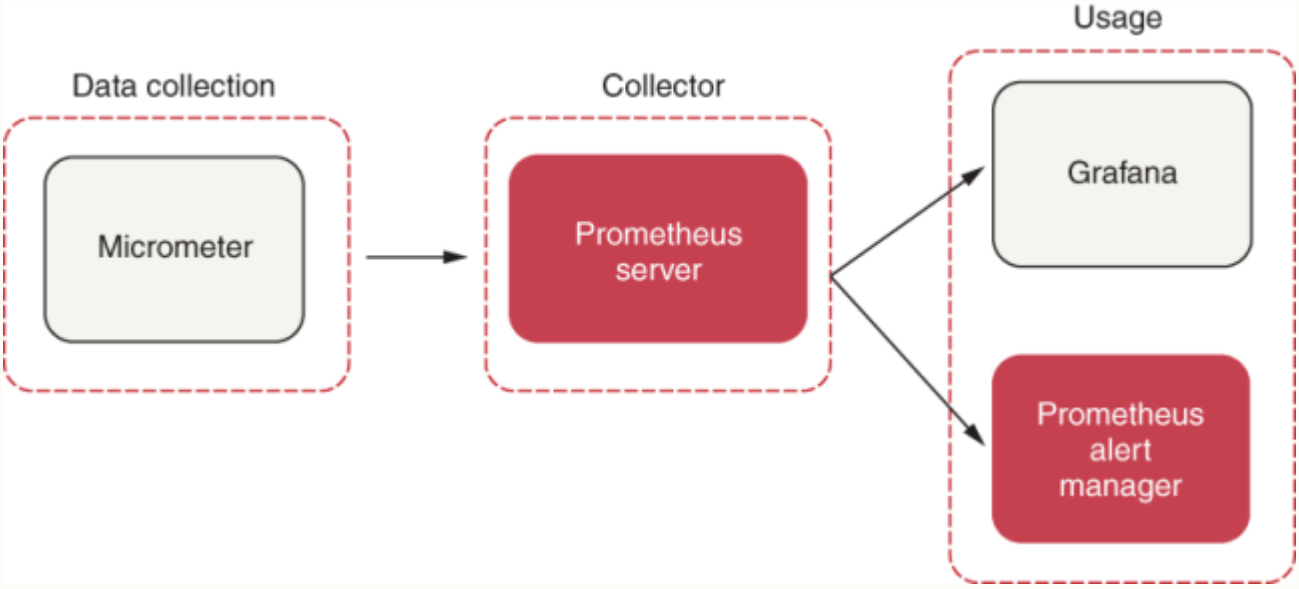


Figure A.7 Interaction between Micrometer, Prometheus, and Grafana

A.6 Logging

Every time you think about logs think about *distributed tracing*. Distributed tracing helps us understand and debug our microservices by pinpointing where a failure occurs in a microservices architecture.

In chapter 11, we explained how to achieve this with Spring Cloud. The traceability of the requests we make within our architecture is another "new" technological (Spring Cloud Sleuth) situation that comes up when speaking about microservices. In monolith architectures, the majority of the requests that reach the application are resolved within the same application. The ones not following this rule were the ones that interacted with a database or some other service. However, in microservices architectures (Spring Cloud, for example), we find an environment composed of several applications, generating a single client request that can go through several applications until it is answered.

How can we follow the path of that request in a microservices architecture? We can associate a unique request identifier that can then be propagated across all the calls and add a central log collection to see all the entries of that client request.

A.7 API gateways

An API gateway is an API REST interface that provides a central access point to a group of microservices and/or defined third-party APIs. In other words, the Spring Cloud API Gateway essentially decouples the interface that clients see from the microservice im-

plementation. This is particularly useful when we want to avoid exposing internal services to external clients. Keep in mind that this system not only acts as a gateway, but you can also add additional features such as these:

- Authentication and authorization (OAuth2)
- Protection against threats (DoS, code injection, and so on)
- Analysis and supervision (who uses the APIs, when, and how)
- Monitoring of incoming and outgoing traffic

Previous chapter
12 Deploying your microservices

Next chapter
appendix B. OAuth2 grant types